Bases de datos basadas en grafos

Guía de Ejercicios

Realizar las consultas que se enumeran a continuación y analizar los resultados. Completar los títulos que faltan.

Consultas

1. Todos los nodos de la base

```
MATCH (n) RETURN (n);
```

Devuelve todos los nodos que estan en la base como nodos de un grafo, es decir, devuelve los nodos como “estructuras” (Sin embargo no devuelve las relaciones)

2. // Count all

```
MATCH (n) RETURN count(n);
```

Toma todos los nodos de la base pero devuelve la cantidad que hay.

3. // Count all relationships

```
MATCH ()-->() RETURN count(*);
MATCH ()-[r]->() RETURN type(r), count(*);
```

La primera devuelve la cantidad de relaciones que hay en la base, sin importar su tipo. La segunda devuelve la cantidad de relaciones que hay pero agrupadas por tipo.

4. // List relationship types

```
CALL db.relationshipTypes();
```

Devuelve todos los tipos de relaciones que hay en la base.

5. // What is related, and how

```
CALL db.schema.visualization();
```

Devuelve un esquema con todas las relaciones posibles (entre todos los nodos que hay en la base).

6. // What kind of nodes exist

```
MATCH (n)
RETURN KEYS(n), COUNT(*) AS s ORDER BY s DESC;
```

Devuelve las distintas “formas” de nodo. Es importante notar que los modos de una misma etiqueta no tienen por que tener las mismas propiedades

7. // Exist?

```
MATCH (a{name: "Keanu Reeves"} ) RETURN a;
```

Devuelve un nodo: Keanu Reeves. Busca por propiedad en todos los nodos, sin etiqueta.

8. Nodos a los que apunta Keanu Reeves (con cualquier tipo de relacion saliente)

```
MATCH (a{name: "Keanu Reeves"} ) -[]-> (f)
RETURN a, f;

MATCH (a) -[]-> (f)

WHERE a.name = "Keanu Reeves"

RETURN a, f;
```

```
MATCH (a{name: "Keanu Reeves"} ) -[r]-> (f)
RETURN a, r, f;
```

Las dos primeras son equivalentes: propiedad en el patron vs WHERE. La tercera agrega r y muestra tambien la relacion.

9. Cantidad de peliculas en las que actuo Keanu Reeves

```
MATCH (a{name: "Keanu Reeves"} ) -[:ACTED_IN]-> (f)
RETURN a, count(f);
```

Matchea todos los nodos que reciben la relacion “ACTED_IN” por parte de KR, devuelve su count y el nodo corresp a KR.

10. Keanu Reeves y las películas en las que actuó (como nodos)

```
MATCH (a{name: "Keanu Reeves"} ) -[:ACTED_IN]-> (f)
RETURN a,  f;
```

Idem arriba pero devuelve todas las f en vez de count.

11. Títulos de las películas en las que actuó Keanu Reeves
```
MATCH (a{name: "Keanu Reeves"} ) -[:ACTED_IN]-> (f)
RETURN f.title;
```

Idem arriba pero devolviendo los titulos en vez de los nodos.

12. Titulos de peliculas estrenadas en 2008
```
MATCH (n:Movie)
WHERE n.released = 2008
RETURN n.title;
```

Matchea todos los nodos de tipo movie donde su prop released es 2008

13. Los primeros 10 nombres de personas en orden alfabetico
```
MATCH (p:Person) RETURN p.name ORDER BY p.name LIMIT 10;
```

Matchea con todas las personas y devuelve todos sus nombres, ordenando en base a su prop name, limitando a 10.

14. Peliculas estrenadas entre 1990 y 2001 de mas nueva a mas vieja.
```
MATCH (m:Movie) WHERE m.released >1990 AND m.released < 2001
RETURN m.title, m.released AS y
ORDER BY y DESC;
```

Las 3 personas de mayor edad (calculada a 2024)
```
MATCH (p:Person)
WHERE p.born IS NOT NULL
RETURN p.name, (2024-p.born) AS s ORDER BY s DESC  LIMIT 3;
```

15. Devuelve a Tom Hanks y las peliculas en las que actuo
```
MATCH (tom:Person {name: "Tom Hanks"})-[:ACTED_IN]->(tomHanksMovies)
RETURN tom, tomHanksMovies;
```

12 películas. tomHanksMovies es solo un nombre de variable: lo que filtra es :ACTED_IN

16. Directores de Cloud Atlas
```
MATCH (cloudAtlas {title: "Cloud Atlas"})<-[:DIRECTED]-(directors)
RETURN directors.name;
```

17. Devuelve los nombres de actores que comparten una pelicula con Tom Hanks
```
MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors)
RETURN coActors.name;
```

18. Devuelve los nombres de las personas relacionadas con Cloud atlas, el tipo de relacion y la relacion.
```
MATCH (people:Person)-[relatedTo]-(:Movie {title: "Cloud Atlas"})
RETURN people.name, Type(relatedTo), relatedTo;
```

19. Devuelve los nodos a 1-4 pasos de Kevin Bacon, por cualquier tipo de relacion (“seis grados de separacion”)
```
MATCH (bacon:Person {name:"Kevin Bacon"})-[*1..4]-(hollywood)
RETURN DISTINCT bacon, hollywood;
```

Hay un ciclo de 4 pasos que vuelve a el por relaciones distintas.

20. Devuelve el camino mas corto entre Kevin Bacon y Meg Ryan

```
MATCH p=shortestPath(
(bacon:Person {name:"Kevin Bacon"})-[*]-(meg:Person {name:"Meg Ryan"}) )
RETURN p;
```

21. Coactores de los coactores de Tom Hanks con los que todavia no trabajo, ordenado por cantidad de conexiones.

```
MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
  (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cocoActors)
WHERE NOT (tom)-[:ACTED_IN]->()<-[:ACTED_IN]-(cocoActors) AND tom <> cocoActors
RETURN cocoActors.name AS Recommended, count(*) AS Strength ORDER BY Strength DESC;
```

22. Devuelve a TH, TC y el nodo y las dos relaciones uqe los conectan. (Como conectarlos a traves de un solo coactor)

```
MATCH (tom:Person {name:"Tom Hanks"})-[:ACTED_IN]->(m)<-[:ACTED_IN]-(coActors),
  (coActors)-[:ACTED_IN]->(m2)<-[:ACTED_IN]-(cruise:Person {name:"Tom Cruise"})
RETURN tom, m, coActors, m2, cruise;
```

23. Devuelve los 10 anios de nacimiento con mas personas.

```
MATCH (p:Person)
RETURN p.born, COUNT(*) AS s ORDER BY s DESC LIMIT 10;
```

24. Devuelve los 10 anios de estreno con menos peliculas

```
MATCH (m:Movie)
RETURN m.released, COUNT(*) AS s ORDER BY s
LIMIT 10;
```

Hay empates para 1 y 2, multiples anios con 1 (o 2) releases.

25. Agregar a cada persona y pelicula una propiedad etiqueta con su nombre o titulo.

```
MATCH (p:Person) SET p.etiqueta = p.name;
MATCH (p:Movie) SET p.etiqueta = p.title;
```